Graph-Based Intrusion Detection: Detecting Network Attacks with Graph Neural Networks and LLM-Generated Alert Explanations

Shiyi Liu (Team Leader)
Dept. of Computer Science
Virginia Tech
lshiyi25@vt.edu

Vighnesh Nair
Dept. of Computer Science
Virginia Tech
vighneshnair@vt.edu

Krishna Inukonda
Dept. of Computer Science
Virginia Tech
kinukond@vt.edu

Gokulakrishnan Ravi
Dept. of Computer Science
Virginia Tech
gravi@vt.edu

***Abstract*—Traditional network intrusion detection systems (NIDS) classify network flows independently, often overlooking communication patterns among hosts. This project develops a graph-based intrusion detection framework using NF-UNSW-NB15-v2 and NF-CICIDS2018-v3. Network flows are represented as attributed edges between IP-address nodes in fixed-size windows of 5,000 flows. We implement an edge-aware graph neural network based on E-GraphSAGE and compare it against Random Forest, XGBoost, and MLP baselines. We also investigate attention-based aggregation and cross-dataset generalization. An LLM explanation layer will generate human-readable alerts from detected malicious flows and their graph context. Evaluation emphasizes macro and per-class F1 scores to account for class imbalance.**

***Index Terms*—intrusion detection, graph neural networks, network security, deep learning, edge classification, domain generalization, large language models**

I. Introduction

Machine-learning-based network intrusion detection systems commonly represent each network flow as an independent feature vector containing information such as duration, packet counts, byte counts, protocol, and port numbers. Although effective, this formulation does not explicitly model relationships among network hosts.

This limitation is important because many attacks are characterized by communication patterns rather than by a single connection. For example, hundreds of connections from one source to many destinations may indicate scanning or brute-force activity, while many sources simultaneously targeting one destination may indicate a denial-of-service attack. Such behaviors naturally form graph structures.

We therefore formulate intrusion detection as an edge-classification problem. For each time window t, we construct a directed multigraph $G_t = (V_t, E_t)$, where V_t contains observed hosts and each $e \in E_t$ represents a network flow with feature vector $\mathbf{x}_e$. The goal is to predict whether each edge is benign or malicious and, for in-domain experiments, its attack category.

The goal of this project is to develop and evaluate a graph-based intrusion detection framework that combines edge-aware graph learning, cross-dataset generalization, and human-readable alert explanations. Our project investigates four questions: whether graph-based modeling can outperform flow-level classifiers; whether attention-based aggregation improves over mean aggregation; how well the learned model generalizes across datasets; and whether an LLM can convert predictions and graph context into useful human-readable alerts.

II. Related Work

Traditional intrusion detection methods commonly classify individual flows using tabular features, with ensemble methods such as Random Forest and XGBoost remaining strong baselines. However, these approaches largely treat flows independently and therefore do not explicitly model relationships among communicating hosts.

Graph neural networks provide a natural alternative because network traffic can be represented as a communication graph. GraphSAGE [3] learns inductive neighborhood aggregation functions, while graph attention networks [4] allow learned weighting of neighboring information. E-GraphSAGE [1] adapts GraphSAGE for network intrusion detection by aggregating incident edge features and classifying network flows using endpoint embeddings. Anomal-E [2] further demonstrates the usefulness of graph-based edge representations for intrusion detection.

A major limitation of existing graph-based NIDS research is that evaluation is often restricted to a single dataset. Cross-dataset comparison is difficult because intrusion datasets typically use different feature schemas. The standardized NetFlow datasets introduced by Sarhan et al. [5] provide a common representation across multiple datasets and therefore enable direct cross-dataset evaluation. Our project combines this standardized representation with edge-centric graph learning, leakage-aware evaluation, and LLM-generated alert explanations.

III. Dataset and Preprocessing

We use two standardized NetFlow datasets: NF-UNSW-NB15-v2 [6] and NF-CICIDS2018-v3 [7]. The datasets con-

tain approximately 2.39 million and 19.49 million flows, respectively. Both are aligned to 39 common numerical features.

Each graph contains 5,000 consecutive network flows. Unique IP addresses represent nodes, while directed edges represent flows with 39-dimensional features and binary/multiclass attack labels. IP addresses and TTL fields are excluded from predictive features.

CICIDS2018-v3 was deduplicated and chronologically split into training (70%), validation (10%), and testing (20%) sets. UNSW-NB15-v2 was split sequentially using its original record order because flow-start timestamps are unavailable.

Selected numerical features undergo logarithmic transformation and standardization using statistics fitted exclusively on the training split. Evaluation emphasizes macro and per-class F1 scores to address class imbalance.

IV. Proposed Approach

A. Baselines

To evaluate whether graph structure provides additional predictive value beyond flow-level features, we plan to compare the proposed GNN against three non-graph baselines trained using the same input features, data splits, and class-weighting strategy:

1) Random Forest.
2) XGBoost.
3) A multilayer perceptron (MLP) with a parameter count chosen to be reasonably comparable to the GNN.

B. Graph Neural Network Model

Our primary graph model is based on the E-GraphSAGE formulation, which is designed for settings in which informative features are associated primarily with edges rather than nodes. At each message-passing layer, a node updates its representation by aggregating information from the incident network-flow edges:

$$h_v^{(k)} = \sigma\left(W^{(k)} \cdot \mathrm{CONCAT}\left(h_v^{(k-1)}, \mathrm{AGG}_{u \in N(v)}\left(h_{uv}^{(k-1)}\right)\right)\right) \tag{1}$$

where h_{uv} denotes the representation associated with the flow edge between nodes u and v, and AGG denotes a permutation-invariant aggregation function such as mean aggregation.

After K message-passing layers, each flow is classified from the learned embeddings of its source and destination nodes, as in the original E-GraphSAGE model:

$$z_{uv} = \mathrm{CONCAT}\left(h_u^{(K)}, h_v^{(K)}\right) \tag{2}$$

The flow features x_{uv} enter the model through the messages aggregated at each layer. The resulting edge representation is passed to a classification head that outputs a probability distribution over the benign class and the available attack categories. As a variant, the original flow features can also be appended to the edge representation, $z_{uv} = \mathrm{CONCAT}(h_u^{(K)}, h_v^{(K)}, x_{uv})$; comparing the two forms is part of the ablation planned for Checkpoint II.

C. Proposed Extension

Beyond the baseline E-GraphSAGE model, we plan to investigate an attention-based aggregation mechanism. Instead of assigning equal importance to all neighboring flows during message passing, the attention mechanism learns weights that allow the model to emphasize more informative or suspicious connections.

This extension is motivated by the observation that network hosts may participate in many benign connections while only a small subset of their neighboring flows are associated with malicious behavior. Mean aggregation may dilute these important signals, whereas attention-based aggregation may allow the model to focus more strongly on the most relevant communication patterns.

We will compare attention-based aggregation against the original mean aggregation used in the baseline model to evaluate whether learned neighbor weighting improves intrusion detection performance, particularly for minority attack classes.

D. LLM Explanation Layer

As an extension to the core intrusion detection pipeline, we plan to add a large language model (LLM) explanation layer that converts model predictions into short, human-readable security alerts. The LLM will not participate in the detection decision itself; instead, it will receive structured information derived from the GNN output and the corresponding local graph context.

For each flow predicted as malicious, the explanation input will include the predicted attack class, prediction confidence, selected flow features, and local graph statistics such as source-host fan-out, the number of distinct destination ports contacted, and the number of similar flows observed within the same time window. This information will be passed to an LLM using a fixed prompt template through the Virginia Tech ARC LLM Gateway.

The generated explanation will summarize why the flow was flagged in plain language while remaining grounded in the supplied model and traffic evidence. To reduce unsupported explanations, the LLM will be instructed to use only the information provided in the structured input and not introduce additional attack evidence. The explanation component will be evaluated qualitatively for factual consistency and clarity.

E. Evaluation Protocol

- **Primary metrics:** macro F1 and per-class F1.
- **Secondary metrics:** precision, recall, confusion matrices, and precision–recall AUC where appropriate.
- **In-domain evaluation:** held-out train/validation/test splits within each dataset, using chronological splitting where reliable timestamps are available.
- **Cross-dataset evaluation:** train on NF-UNSW-NB15-v2 and test on NF-CICIDS2018-v3, and vice versa, using binary benign-versus-malicious classification to account for differences in attack categories.
- **Ablation studies:** compare graph-based versus flow-level models and attention-based versus mean aggregation.

Additional experiments on flow-window size and class weighting will be considered if time permits.

- **Qualitative evaluation:** manual review of a sample of LLM explanations for factual consistency with the underlying flow.

V. Checkpoint I Progress: Data Preparation and Graph Construction (Shiyi Liu)

A. Completed Work

Implemented preprocessing pipelines for NF-UNSW-NB15-v2 (2.39 million flows) and NF-CICIDS2018-v3 (19.49 million flows). Both datasets were aligned to 39 common numerical features using logarithmic transformations and training-only standardization. CICIDS was deduplicated and chronologically split into training (70%), validation (10%), and testing (20%) sets. UNSW was split sequentially using its original record order.

Constructed directed communication graphs using fixed-size windows of 5,000 flows. Unique IP addresses represent nodes, while edges contain 39-dimensional features and binary/multiclass attack labels. Generated 476 UNSW graphs and 3,896 CICIDS graphs (4,372 total). Verified graph counts and validated the structure and numerical integrity of representative graphs.

B. Limitations and Next Steps

UNSW lacks flow-start timestamps, preventing verification of chronological ordering. Full graph-level validation and leakage auditing remain pending. Next steps include further data integrity checks and supporting GNN training and cross-dataset evaluation.

VI. Checkpoint I Progress: Flow-Level Baselines and Evaluation (Gokulakrishnan Ravi)

A. Integration with Part 1

The objective of the non-graph baselines is to quantify how well network attacks can be identified from individual flow attributes before incorporating host communication structure. We build directly on the data preparation described in Section V, without modifying its preprocessing or graph-construction code. A common loader reads the existing graph `.npz` files, extracts the 39-dimensional `edge_attr` vectors and ten-class `edge_attack` labels, and preserves the original graph-based partitions. The `edge_index` connectivity is intentionally excluded from the baseline predictors. The original transformations and feature order are therefore retained while the graph information is withheld.

For NF-UNSW-NB15-v2, the training, validation, and test partitions contain 334, 47, and 95 graphs, respectively, corresponding to 1,670,000, 235,000, and 475,000 flows. To establish an initial working pipeline at manageable cost, we used the existing seeded, class-aware sampler to select 20,000 training flows for each model, while evaluating on all 235,000 validation flows. The sampled training labels include all ten classes, including the rarest categories.

B. Implemented Models

Random Forest (RF): The initial ensemble comprises ten decision trees with maximum depth 10 and minimum leaf size two. Balanced subsample weighting is applied during fitting. This compact configuration provides an inexpensive reference for nonlinear flow-level classification.

XGBoost: The gradient-boosted tree classifier uses a multi-class probability objective, 50 estimators, maximum depth five, and learning rate 0.1. Histogram-based tree construction is used for CPU execution. Balanced observation weights allow attack classes with limited representation to influence the fitted objective.

Multilayer Perceptron (MLP): The PyTorch classifier has layer widths $39 \to 64 \to 32 \to 10$, ReLU activations, and dropout of 0.2. It is trained with Adam (learning rate 10^{-3}), class-weighted cross-entropy, and mini-batches of 256 flows, for at most ten epochs. Early stopping uses a patience of three epochs, with the best validation-loss weights restored before evaluation. This neural baseline tests nonlinear feature learning without message passing.

C. Imbalance Handling and Evaluation

The original UNSW training partition contains 1,625,729 benign flows out of 1,670,000, making overall accuracy potentially misleading. The 20,000-flow pilot sample contains 19,401 benign and 599 attack flows. RF uses balanced subsample weights, whereas XGBoost and the MLP use class frequencies from the sampled training labels to form

$$w_c = \frac{N}{K n_c}, \tag{3}$$

where N is the sampled training count, $K = 10$, and n_c is the number of sampled flows in class c. For XGBoost these are per-flow training weights; for the MLP they weight the cross-entropy loss.

We also implemented shared evaluation utilities for accuracy, macro and weighted precision, recall and F1, per-class statistics, and confusion matrices. Macro F1 receives particular attention because it weights all ten classes equally. Binary intrusion-detection metrics are obtained by collapsing multiclass predictions to attack versus benign, using the dataset mapping in which Benign is class 2.

D. Checkpoint I Results

All three models completed a single-seed pilot using a 20,000-flow training budget (seed 42) and the complete UNSW validation set. Table I reports the resulting metrics.

XGBoost obtained the highest pilot macro F1 (0.4331) and binary attack F1 (0.8555). The MLP reached a binary F1 of 0.8345 but a lower macro F1 of 0.2972, indicating difficulty differentiating among attack categories. For example, the MLP achieved F1 scores of 0.0085 for DoS and 0.0202 for Worms. These results demonstrate that the three pipelines can train and evaluate on prepared graph data; they do not establish a final model ranking.

TABLE I
PILOT BASELINE RESULTS ON NF-UNSW-NB15-V2 VALIDATION FLOWS ($n = 235{,}000$)

Metric	RF	XGBoost	MLP
Accuracy	0.9551	0.9550	0.9364
Macro F1	0.3755	**0.4331**	0.2972
Weighted F1	0.9536	**0.9621**	0.9429
Binary attack F1	0.7977	**0.8555**	0.8345

E. Checkpoint II Plan and Limitations

The present experiments use only a fraction of the available training flows, one random seed, and model-specific weighting and hyperparameter settings. Although their sampling budgets and recorded class counts match, identical selected flow indices have not been independently verified. Moreover, the baseline results above use the validation partition, while the initial E-GraphSAGE results in Table II use the held-out test partition. Their scores must not be interpreted as a controlled graph-versus-non-graph comparison.

Checkpoint II will increase the training budget; compare weighted and unweighted settings; repeat experiments across seeds; and analyze minority-class errors. We will extend the baselines to NF-CICIDS2018-v3, complete standardized held-out test evaluation, and coordinate with the GNN experiments to compare predictions on the same partitions using the shared metrics. These studies will determine whether graph structure adds predictive value beyond individual flow features.

VII. CHECKPOINT I PROGRESS: GRAPH NEURAL NETWORK MODEL (VIGHNESH NAIR)

A. Implementation

E-GraphSAGE [1] is implemented in PyTorch Geometric following Eq. (1): each layer forms a message $m_{uv} = W_{\text{msg}}[h_u \| x_{uv}]$ for every incident flow and updates the host as $h_v = \text{ReLU}(W_{\text{apply}}[h_v \| \text{AGG}_u\, m_{uv}])$ with mean aggregation. Node inputs are constant, so all information comes from the flows. Each flow also sends a message in the reverse direction, marked by a 0/1 direction flag, so that a host that only sends traffic (for example, a scanner) still aggregates its flows. Each flow is classified from the embeddings of its two endpoints, $[h_u \| h_v]$, as in the original model.

Training uses Adam (learning rate 10^{-3}), two layers with 128 hidden units, dropout 0.2, early stopping on the validation set, and a class-weighted cross-entropy loss to handle the strong class imbalance. Every run is seeded and defined by a saved configuration file, and metrics are logged every epoch so that any reported number can be reproduced. The model takes the windowed graphs described in Section III directly as input, and the code rejects any input in which IP addresses, timestamps, or IP-header TTL appear among the features. The implementation has been tested end to end on the GPU server, and runs are reproducible for a fixed seed.

TABLE II
INITIAL E-GRAPHSAGE RESULTS (NF-UNSW-NB15-V2, TEST SET)

Metric	Value
Macro F1 (10 classes)	0.292
Weighted F1	0.942
Accuracy	0.933
Binary benign-vs-attack F1	0.874
Per-class F1	
Benign	0.988
Fuzzers	0.574
Generic	0.524
Exploits	0.356
Analysis	0.182
Backdoor	0.157
Reconnaissance	0.074
DoS	0.030
Shellcode	0.025
Worms	0.007

TABLE III
GNN TASK STATUS

Task	Status
E-GraphSAGE implementation	Done
Training pipeline (seeding, configs, logging)	Done
Initial results (UNSW, 1 seed)	Done
Full multi-seed results (both datasets)	Checkpoint II
Attention-based aggregation	Checkpoint II
Hyperparameter and class-weight tuning	Checkpoint II
Ablation studies	Checkpoint II
Cross-dataset experiments	Checkpoint II

B. Initial Results

As a first evaluation, E-GraphSAGE with mean aggregation was trained on NF-UNSW-NB15-v2 (334 training graphs) with a single seed and default hyperparameters, and evaluated on the held-out test windows (475,000 flows). Table II reports the results; F1 is computed with scikit-learn and will be re-reported through the shared evaluation module.

The model separates benign traffic and the more frequent attack types well, but the macro F1 of 0.292 is held down by the rarest classes (Worms has only 71 training flows), which motivates the class-weighting and hyperparameter study planned for Checkpoint II. These are initial single-seed results on one dataset; experiments on NF-CICIDS2018-v3 and multi-seed comparisons will follow.

C. Status of GNN Tasks

D. Plan for Checkpoint II

1) Report multi-seed E-GraphSAGE results on both datasets against the baselines, using the shared macro and per-class F1 module.
2) Evaluate attention-based aggregation against mean aggregation over three seeds.
3) Tune class weighting, depth, hidden size, and learning rate on the validation set.

4) Run the graph-versus-flow-level ablation and a per-class error analysis.
5) Train on NF-CICIDS2018-v3 and run cross-dataset experiments in both directions.

VIII. Checkpoint I Progress: Explanation, Visualization, and Demo (Krishna Inukonda)

A. Project Website and Repository Infrastructure

The team's public project website was created and deployed on GitHub Pages with open read access to host the project proposal, team member profiles, and checkpoint reports. We also participated in team design decisions to finalize the cross-dataset evaluation protocol (retaining in-domain multi-class training while evaluating cross-dataset transfer as binary benign-versus-attack), remove `MIN_TTL` and `MAX_TTL` to prevent host-identification leakage, and migrate to timestamp-ordered NF-CSE-CIC-IDS2018-v3.

B. Status of Explanation and Visualization Tasks

TABLE IV
Explanation, Visualization, and Demo Task Status

Task	Status
Project website deployment and maintenance	Done
Downstream data contract & interface design	Done
LLM prompt template & ARC Gateway prototype	Checkpoint II
Subgraph context extraction from GNN outputs	Checkpoint II
Qualitative evaluation of LLM explanations	Checkpoint II
Visualization dashboard & final demo	Final Report

C. Plan for Checkpoint II

1) Implement the local subgraph statistics extractor on top of the finalized 5,000-flow graph objects and saved E-GraphSAGE predictions.
2) Connect the Virginia Tech ARC LLM Gateway and build the fixed prompt template to generate short, grounded security alerts for malicious flows.
3) Perform initial qualitative checks on generated alerts to verify factual consistency with the input flow features and graph context.
4) Begin prototyping the visualization dashboard for the final demonstration.

IX. System Design

The system consists of four stages: preprocessing and graph construction, model training, inference and explanation, and visualization. Models will be implemented using PyTorch and PyTorch Geometric and trained using Virginia Tech computing resources.

The overall pipeline is:

$$\text{NetFlow Data} \rightarrow \text{Graph Construction} \rightarrow \text{GNN/Baselines} \rightarrow \text{Attack Prediction} \rightarrow \text{LLM Explanation}$$

If time permits, a lightweight dashboard will visualize selected communication graphs, malicious-flow predictions, confidence scores, evaluation results, and LLM-generated explanations.

References

[1] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022.

[2] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, 2022.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[5] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications*, Springer, 2021.

[6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015.

[7] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

TABLE V
Task Assignment

Member	Responsibilities
Shiyi Liu	Dataset preparation, preprocessing, temporal splitting, and graph construction.
Gokulakrishnan Ravi	Random Forest, XGBoost, MLP baselines, and the shared evaluation metrics (macro and per-class F1).
Vighnesh Nair	E-GraphSAGE, attention-based extension, hyperparameter tuning and class weighting, ablations, and cross-dataset experiments.
Krishna Inukonda	LLM explanation module, dashboard, project website, and demonstration video.
All members	Proposal/report writing and checkpoint/final presentations.

TABLE VI
Project Schedule

Period	Milestone
Sep 15–Oct 6	Dataset preprocessing, graph construction, and baseline models.
Oct 7–20	Initial E-GraphSAGE implementation and evaluation.
Oct 21–Nov 10	Attention extension, cross-dataset experiments, and LLM prototype.
Nov 11–Dec 11	Ablation studies, dashboard, final experiments, presentation, and final report.